

Guía de Pruebas — Tower Defense Estocástico

Paso a paso para correr **todas** las verificaciones del proyecto y comprobar que el modelo de colas funciona, es reproducible y concuerda con la teoría. Cada prueba indica **qué tema de la materia** demuestra.

Equipo: Luciano Esteban · Thael Anaya · Martín Blasson · Luca Sucrí

Profesor: Sergio Sirotinsky

Modelo: (M/M/c)(FIFO/K/∞) + temperatura

CONTENIDO

1. El modelo en una frase
2. Resumen de verificaciones
3. Paso 1 — Preparar el entorno
4. Paso 2 — Tests del modelo (17)
5. 🔍 Recorrido del código SimPy
6. Paso 3 — Simulación y export
7. Paso 4 — Auditoría del contrato
8. Paso 5 — Análisis (Módulo B)
9. Paso 6 — Experimentos (Módulo B+)
10. Paso 7 — Frontend en Godot
11. Temas de la materia ↔ pruebas



El modelo en una frase

Cada **torre** = **servidor** de un sistema M/M/c (cola FIFO única); cada **enemigo** = **cliente**; una **fuga** = **bloqueo** por capacidad finita K. La **temperatura** (variable continua que decae exponencial hacia la ambiente —ley de enfriamiento de Newton—) apaga torres sobrecalentadas, reduce la capacidad efectiva y **rompe el supuesto markoviano**: por eso **simular** aporta sobre la fórmula cerrada. El objeto de estudio **no es el juego**, sino la **validación del modelo de colas**.



Resumen de verificaciones

Esto es lo que vas a comprobar. Si todo da verde, el proyecto está sano de punta a punta.

Tests del modelo

17 / 17 OK

Contrato output.json

v1.0 válido

Ley de Little (consistencia)

error $\approx$ 1.2 %

Óptimo económico

$c^* = 3$ torres

Fuga simulada vs analítica

4.0 % vs 0.16 %

Reproducibilidad

byte-idéntico

Front (Godot 4.3)

compila + render

Export Web (HTML5)

PWA generado

Tiempo total estimado: ~5 min para todo el backend (pasos 1–6) en una máquina modesta. El front (paso 7) depende de tener Godot instalado.

1 Preparar el entorno

Objetivo: crear el entorno virtual de Python e instalar las dependencias (`simpy`, `numpy`, `pandas`, `matplotlib`). Una sola vez.

```
# Repo del backend (toda la matemática vive acá)
cd repositorios/Tower-Defense-Estocastico-back
python -m venv .venv
# Activar el venv:
source .venv/Scripts/activate      # Windows (Git Bash)
source .venv/bin/activate          # Linux / macOS
pip install -r requirements.txt
```

TERMINAL

Si ya tenés el `.venv` creado, alcanza con **activarlo**. En Windows también podés usar directamente

.venv/Scripts/python.exe sin activar.

2 Tests del modelo — el corazón de la validación

17 tests que comprueban, con tolerancias matemáticas, que cada pieza hace lo que la teoría predice. **No requieren pytest** (corren solos).

Objetivo: que las 17 afirmaciones del modelo se cumplan.

```
python tests/test_modelo.py # o, si tenés pytest: python -m pytest -q
```

TERMINALSALIDA ESPERADA

```
PASS test_exponencial_inversa_formula
PASS test_exponencial_media_empirica
PASS test_gcl_reproducible
PASS test_mm1_peaje
PASS test_mmc_estabilidad
PASS test_mmc_reduce_a_mm1
PASS test_mmck_K_igual_c_es_erlang_b
PASS test_mmck_bloqueo_en_rango
PASS test_no_estacionario_respetar_tramos
PASS test_prioridad_fuerte_espera_menos
PASS test_prioridad_work_conserving
PASS test_sim_balance_enemigos
PASS test_sim_concuerda_con_analitico_sin_temperatura
PASS test_sim_little
PASS test_sim_reproducible
PASS test_tipos_misma_media_distinta_varianza
PASS test_extension_no_cambia_default
```

17/17 tests OK

Qué valida cada grupo de tests

Grupo	Tests	Qué demuestra
-------	-------	---------------

Aleatoriedad

`exponencial_inversa_formula,`
`exponencial_media_empirica, gcl_reproducible`

La transformada inversa genera $\text{Exp}(\lambda)$ correcta (media empírica $\approx 1/\lambda$) y el generador congruencial es reproducible.

Colas (analítico)

`mm1_peaje, mmc_estabilidad, mmc_reduce_a_mm1,`
`mmck_bloqueo_en_rango, mmck_K_igual_c_es_erlang_b`

Las fórmulas cerradas son correctas: M/M/1 reproduce el ejemplo del apunte, M/M/c con $c=1 \equiv \text{M/M/1}$, M/M/c/K calcula bloqueo y con $K=c$ da Erlang B.

Simulación

`sim_little, sim_balance_enemigos, sim_reproducible,`
`sim_concuerda_con_analitico_sin_temperatura`

La simulación cumple la **Ley de Little**, conserva enemigos ($\text{spawn} = \text{kill} + \text{fuga} + \text{en sistema}$), es reproducible con semilla, y **sin temperatura coincide con Erlang C** (motor validado).

Extensiones

`no_estacionario_respeta_tramos,`
`tipos_misma_media_distinta_varianza,`
`prioridad_fuerte_espera_menos,`
`prioridad_work_conserving,`
`extension_no_cambia_default`

Las extensiones opt-in funcionan y **no alteran** el comportamiento por defecto.

🎓 **Tema clave:** el test `sim_concuerda_con_analitico_sin_temperatura` es el que **valida el simulador contra la teoría**: si apagamos la temperatura, la simulación tiene que dar lo mismo que la fórmula de Erlang C. Que coincidan es la prueba de que el motor es correcto.



Recorrido del código SimPy — dónde y qué mirar

Toda la simulación vive en `simulation.py`. Estos son los **6 lugares clave**: qué concepto de la materia implementa cada uno, el fragmento real y **qué mirar**. Las referencias son `archivo` : línea sobre la versión actual del repo.

Eventos discretos

El reloj, la cola y los streams de azar

`simulation.py` : 138–151

```
self.env = simpy.Environment()          ← reloj + lista de eventos futuros (FEL)
self.queue = simpy.PriorityStore(...) if sc.priority else simpy.Store(...) # cola FIFO o con
prioridad
self.towers = [Tower(i, sc) for i in range(sc.c)] # c servidores
self.stream_arr = ExponentialStream(sc.seed)      # azar de arribos (stream propio)
self.stream_srv = ExponentialStream(sc.seed + 10_007) # azar de servicio (stream propio)
```

Qué mirar: `simpy.Environment()` ES el **algoritmo de lista de eventos** de la Unidad III: el reloj salta de evento a evento, no en pasos fijos. `Store` es la **cola FIFO única**; cambiarla por `PriorityStore` da la disciplina con **prioridad**. Un stream de azar por fuente = **números aleatorios comunes** (misma semilla $\Rightarrow$ misma corrida, y comparaciones justas entre escenarios).

Llegadas Poisson + bloqueo

¿Cuándo aparece cada enemigo?

`simulation.py` : 215–241

```
def arrivals(self):
    while True:
        yield self.env.timeout(self.stream_arr.sample(self._current_lam())) ← Exp( $\lambda$ ): tiempo
        hasta el próximo arribo
        ...
        if self.sc.K > 0 and self._in_system()  $\geq$  self.sc.K: ← sistema lleno (capacidad K)
            self.res.leaked += 1                                # BLOQUEO = la "fuga" del juego
            self._ev(now, "leak", ...); continue
        self.queue.put(enemy)                                   # si hay lugar, entra a la cola
```

Qué mirar: la línea del `timeout(Exp( $\lambda$ ))` es el **criterio de cuándo llega cada goblin**: un **proceso de Poisson** de tasa λ (tiempos entre arribos exponenciales, sin memoria). El `if ...  $\geq$  K` es el **sistema con pérdida**: el cliente que llega con todo ocupado se pierde — esa es la fuga.

Servidor + servicio

La torre atiende (FIFO, tiempo Exp(μ))

`simulation.py` : 244–271

```
def tower_proc(self, tower):
    while True:
        item = yield self.queue.get()      ← FIFO: toma el siguiente (espera si la cola está vacía)
        ...
        ts = self.stream_srv.sample(self.sc.mu * enemy["mu_factor"]) ← tiempo de servicio Exp( $\mu$ )
        yield self.env.timeout(ts)         # la torre "dispara" durante ts
        ...
        self._ev(now, "kill", ...)         # enemigo eliminado
```

Qué mirar: cada torre es un **servidor**; `queue.get()` implementa la **disciplina FIFO** (y espera sola cuando no hay trabajo); $\text{Exp}(\mu)$ es el **tiempo de servicio**. El `mu_factor` permite enemigos "fuertes/débiles" (V.A. discreta de tipo) sin tocar el motor.

Variable continua (EDO)

La temperatura que "rompe la fórmula"

simulation.py : 59-68 · 274-285

```
def temp_at(self, now):
    # variable CONTINUA de estado
    if self._heating: return self.temp + k_heat * dt      ← rampa lineal al disparar
    return Ta + (self.temp - Ta) * math.exp(-k_cool * dt) ← decae exponencial hacia la ambiente (ley de Newton)
    ...
    if tower.temp ≥ self.sc.T_max:      # se sobrecalentó
        tower.go_cooldown(now)          ← la torre deja de atender: cae la capacidad efectiva
        yield self.env.timeout(cd)      # espera a enfriarse hasta T_resume
```

Qué mirar: la temperatura es la **variable continua**, integrada por una **ley de enfriamiento exponencial** (de Newton) en forma cerrada por tramos. Al entrar en `cooldown` la torre deja de ser un servidor disponible → **rompe el supuesto markoviano** (c servidores siempre activos) → la M/M/c queda como **cota** y por eso simular aporta.

Estimadores · Ley de Little

De dónde salen L_q y L

simulation.py : 167-182

```
def _integrate(self):
    # estimación por ÁREA BAJO LA CURVA
    dt = now - self._last_int_t
    self.res.q_area += len(self.queue.items) * dt      ←  $\int (\text{largo de cola}) dt \rightarrow L_q$ 
    self.res.sys_area += self._in_system() * dt        ←  $\int (\text{clientes en sistema}) dt \rightarrow L$ 
```

Qué mirar: L y L_q **no** salen de una fórmula, sino de **integrar el estado en el tiempo** ($L_q = \int \text{cola } dt / T$) — el estimador estadístico de la simulación. Que después se cumpla **Little** ($L = \lambda \cdot W$) es la prueba de consistencia interna (test `sim_little`, error $\approx 1.2\%$).

El motor Leer `run()` = leer el algoritmo de lista de eventos

`simulation.py` : 304-309

```
def run(self):
    self.env.process(self.arrivals())           # entidad: llegadas
    for tw in self.towers: self.env.process(self.tower_proc(tw)) # entidades: torres
    self.env.process(self.monitor())           # foto del estado cada dt_sample
    self.env.run(until=self.sc.sim_time)       ← procesa la FEL en orden hasta el horizonte
T
```

Qué mirar: registrarás las entidades como **procesos** y `env.run(until=T)` extrae y procesa los eventos en orden cronológico hasta el horizonte. Eso es, literalmente, el **esquema de lista de eventos futuros** de la materia, sin escribir el `while/heap` a mano.

🔗 **El azar que alimenta los timeout:** los $\text{Exp}(\lambda) / \text{Exp}(\mu)$ se generan por **transformada inversa** en `prng.py` : 53 $\rightarrow \text{exponencial_inversa}(R, \lambda) = -(1/\lambda) \cdot \ln(1-R)$, a partir de un generador **congruencial** reproducible (`prng.py` : 22). Las fórmulas cerradas contra las que se valida (Erlang C / Erlang B) viven en `analytical.py`.

3 Simulación + export del `output.json`

Objetivo: correr el escenario por defecto, barrer $c = 1 \dots 6$ torres, y exportar el `output.json` (el contrato que consume el frontend). Incluye una **auto-auditoría** del esquema antes de guardar.

```
python main.py
# Variantes:
python main.py --c 4 --lam 0.5 --mu 0.25 --sim 1800 --seed 7
python main.py --no-sweep --out otra.json
```

TERMINAL

```
TOWER DEFENSE ESTOCÁSTICO – Módulo A (Simulación SimPy)
Modelo (M/M/c)(FIFO/K/inf): c=3 K=10
```

SALIDA ESPERADA

```
lambda=0.4 mu=0.25 rho=0.533 sim_time=1200.0s seed=42
Enemigos: spawn=455 kill=434 fuga=18 base_hp=82
Corriendo barrido de torres c=[1, 2, 3, 4, 5, 6] ...
[OK] Auto-auditoría del esquema superada (contrato v1.0).
[OK] Exportado 'output.json' (1815 eventos, 2400 muestras)
```

🎓 **Tema:** simulación por **eventos discretos** con SimPy (algoritmo de lista de eventos) y **reproducibilidad** por semilla. $\rho = \lambda / (c \cdot \mu) = 0.533 < 1 \rightarrow$ sistema estable.

4 Auditoría del contrato de datos

Objetivo: verificar que el `output.json` cumple las 8 invariantes del contrato v1.0 (el frontend depende de esto). `main.py` ya la corre, pero podés lanzarla suelta.

```
python validate_schema.py output.json
```

TERMINAL

```
[OK] 'output.json' cumple el contrato v1.0.
```

SALIDA ESPERADA

🎓 **Tema:** desacople **backend** ↔ **frontend** mediante un contrato versionado. El backend calcula offline; Godot solo reproduce. **Cero acoplamiento en tiempo real.**

5 Análisis — Módulo B (gráficos + conclusiones)

Objetivo: leer el `output.json`, generar 5 figuras en `figs/` e imprimir las conclusiones del análisis.

```
python analysis.py
python analysis.py otra.json --outdir figs2 # sobre otro archivo
```

TERMINAL

```
[Ley de Little]  $\lambda_{\text{eff}} \cdot W = 0.362 \cdot 7.350 = 2.658$  vs  $L_{\text{sim}} = 2.626$ 
                error relativo = 1.2% (OK)
```

SALIDA ESPERADA

```
[Capacidad]  $c_{\text{min}} = 2$  ; 1ª torre sobre  $c_{\text{min}}$  captura 89% de la mejora
```


ΔLq por torre: [2.53, 0.25, 0.05, 0.01]

→ RENDIMIENTO MARGINAL DECRECIENTE

[Decisión económica] $c * (\text{torres} + \text{fugas}) = 3$

[Temperatura] Fuga simulada (4.0%) vs analítica (0.2%): brecha = +3.8%

→ la explica la indisponibilidad por temperatura (M/M/c/K no la modela)

Gráficos: `figs/01_series_colas` · `02_temperatura` · `03_utilizacion`

· `04_rendimiento_marginal` · `05_costo_optimo`

🎓 **Temas:** Ley de Little (consistencia), **rendimiento marginal decreciente** al sumar servidores, **óptimo económico** (costo capacidad + costo fugas), y la **brecha simulación–teoría** que justifica simular.

6 Experimentos avanzados — Módulo B+

Objetivo: los 4 estudios que profundizan el análisis. Generan figuras en `figs_exp/`.

```
python experiments.py
python experiments.py --reps 20 --only ci # más réplicas, un solo estudio
```

TERMINAL

Estudio	Qué hace	Hallazgo
1. Réplicas + IC 95%	Repite con N semillas y arma intervalos de confianza.	$c^* = 3$ robusto (no depende de una corrida afortunada).
2. No estacionario	Arribos en oleadas $\lambda(t)$ (valle/pico).	Dimensionar al promedio → fuga 42% (base destruida); al pico → 10%. Dimensionar al pico.
3. Tipos de enemigo	V.A. discreta con igual media de servicio, distinta varianza.	A igual media, más varianza del servicio ⇒ fuga +0.94 pp (más variabilidad ⇒ más cola).
4. Prioridad	Atender al "fuerte" primero (disciplina con prioridad, sin desalojo).	Wq del fuerte 4.24→2.15 s; el Wq total y la fuga no cambian: reparte la espera entre clases.

🎓 **Temas: reducción de varianza** (réplicas, números aleatorios comunes), sistemas **no estacionarios**, **variables aleatorias discretas**, y **disciplinas de cola** con prioridad.

7 Frontend en Godot — reproducción visual

El front **no calcula nada**: lee el `output.json` y lo reproduce en 2D. Verificar el render es opcional (requiere Godot 4.3+).

A) Abrir y correr en el editor

1. Abrir Godot 4.3+, botón **Import** → elegir `repositorios/Tower-Defense-Estocastico-front/project.godot`.
2. Confirmar que `output.json` está en la raíz del proyecto (copiado del backend).
3. Presionar **F5** (Play). Controles: **Espacio** play/pausa · **</>** velocidad · **R** reiniciar.

Deberías ver: 3 torres con su radio de rango, coloreadas **azul→rojo** según temperatura; enemigos amarillos sobre el camino; líneas de fuego torre→enemigo; la base con barra de vida; y el HUD con λ , μ , ρ , fuga, Wq , Lq .

B) Exportar a Web (HTML5)

```
# con Godot en el PATH y los export templates de 4.3 instalados:  
godot --headless --export-release "Web" builds/web/index.html
```

TERMINAL

Para hostear gratis (GitHub Pages / itch.io) el preset ya trae el **service worker PWA** que inyecta los headers de aislamiento de origen (COOP/COEP). Probar local requiere servir `builds/web/` con esos headers.

✅ **Estado verificado**: el proyecto compila sin errores, corre 600 frames sin fallos, y el render se confirmó con una captura real del juego (export web + canvas WebGL).



Temas de la materia ↔ dónde se demuestran

Esta es la traza explícita de **qué entendemos** y **cómo lo probamos** en el código. Útil para defender el TP: cada concepto tiene una prueba ejecutable que lo respalda.

Tema de la materia	Dónde vive en el código	Prueba que lo demuestra
U2 Generación de números pseudoaleatorios (congruencial lineal)	<code>prng.py</code> · <code>CongruentialGenerator</code>	<code>test_gcl_reproducible</code> (Paso 2)
U2 Transformada inversa para V.A. continuas	<code>prng.py</code> · <code>exponencial_inversa</code> $x = -(1/\lambda) \cdot \ln(R)$	<code>test_exponencial_inversa_formula, ...media_empirica</code> (Paso 2)
U2 Variables aleatorias (Exponencial; discreta)	<code>prng.py</code> · <code>ExponentialStream</code> ; tipos de enemigo en <code>config.py</code>	<code>test_tipos_misma_media_distinta_varianza</code> · Estudio 3 (Paso 6)
U3 Notación de Kendall-Lee · M/M/1	<code>analytical.py</code> · <code>mmc(...)</code> (caso $c=1$)	<code>test_mm1_peaje, test_mmc_reduce_a_mm1</code> (Paso 2)
U3 M/M/c (Erlang C) y estabilidad $\rho < 1$	<code>analytical.py</code> · <code>mmc, _erlang_c</code>	<code>test_mmc_estabilidad</code> · barrido en Paso 3
U3 M/M/c/K (capacidad finita) · probabilidad de bloqueo	<code>analytical.py</code> · <code>mmck</code> → "fuga"	<code>test_mmck_bloqueo_en_rango, ...K_igual_c_es_erlang_b</code> (Paso 2)
U3 Ley de Little (L = $\lambda \cdot W$)	estimadores en <code>simulation.py</code> + chequeo en <code>analysis.py</code>	<code>test_sim_little</code> · error $\approx 1.2\%$ (Pasos 2 y 5)
SIM Simulación por eventos discretos (lista de eventos)	<code>simulation.py</code> · <code>TowerDefenseSim</code> (SimPy = reloj + FEL)	<code>test_sim_balance_enemigos, test_sim_reproducible</code> (Pasos 2 y 3)

SIM Variable continua de estado (EDO)	simulation.py · Tower — enfriamiento exponencial (Newton) por tramos	Figura 02_temperatura · brecha de fuga (Paso 5)
VAL Validación: simulación vs. fórmula cerrada	sim sin temperatura $\stackrel{?}{=}$ Erlang C	test_sim_concuerda_con_analitico_sin_temperatura (Paso 2)
VAL Cuándo simular aporta sobre el modelo analítico	la temperatura rompe Markov → fuga sim $\gg$ analítica	Conclusión "Temperatura" (Paso 5)
ADV Reducción de varianza · intervalos de confianza	experiments.py — réplicas + streams independientes	Estudio 1 — IC 95% (Paso 6)
ADV Sistemas no estacionarios ($\lambda(t)$)	config.py · lam_schedule	Estudio 2 · test_no_estacionario_respeto_tramos (Pasos 6 y 2)
ADV Disciplinas de cola con prioridad	config.py · priority + simpy.PriorityStore	Estudio 4 · test_prioridad_* (Pasos 6 y 2)
ADV Decisión / optimización económica	curva costo(c) en U → c*	Figura 05_costo_optimo (Paso 5)

Tower Defense Estocástico · TP de Simulación de Sistemas · UCEMA · Ing. en Informática

Repos: [rag-karpathy](#) (wiki) · [back](#) (SimPy) · [front](#) (Godot) · [presentaci-n](#) (deck) · Las cifras son de la corrida base ($\lambda=0.4$, $\mu=0.25$, $c=3$, $K=10$, $\text{seed}=42$).